Semantic Information Gain for Robust Human-in-the-Loop Program Synthesis: A Partial Implementation and Controlled Evaluation

Nguyễn Thành Đạt, Đặng Bá Ty, Nguyễn Tiến Vũ Bảo
Faculty of Computer Science and Engineering
Ho Chi Minh City University of Technology (HCMUT), VNU-HCM, Vietnam
Supervisor: TS. Võ Thanh Hùng
Course CO3101: Integrated Artificial Intelligence Project, Semester 261

***Abstract*—Ambiguous natural-language specifications admit several plausible programs. SEMSYNTH addresses this problem through semantic clarification, information gain, Bayesian belief revision, and hypothesis management. This course project preserves the full architectural design while implementing a small subset: candidate generation adapters, bounded execution probes, binary output questions, and hard filtering. We explain how binary partition entropy follows from the original information-gain objective under uniform priors and deterministic observations. On 32 controlled synthetic cases from eight related template families, the first-candidate and first-valid selectors each pass all retained evaluation tests on 13 cases (40.625%), while balanced clarification passes on 32 cases, using 1.4688 questions per case. Saved pilot artifacts for two model identifiers each contain eight synthetic tasks; balanced selection passes all retained tests on both pilots, versus seven and six passing tasks for the respective noninteractive selectors. Feedback comes from a noiseless reference-program oracle. The 15-task HumanEval/MBPP-derived pilot catalog is audited separately and is not the measured benchmark. The results support selection behavior within these saved pools; they do not establish general program correctness, human effort reduction, noisy-feedback robustness, or large-scale performance. Source, saved pools, verification logs, and LaTeX files accompany the report.**

***Index Terms*—Program synthesis, human-in-the-loop, information gain, clarification, partial implementation, reproducible evaluation**

I. INTRODUCTION

Program synthesis seeks executable programs from high-level specifications. A short description can leave output order, duplicate treatment, boundary inclusion, or case sensitivity unspecified. For example, "remove duplicates from a list" is compatible with preserving first occurrences or returning sorted unique values. Syntax checks cannot resolve this difference because both programs may be valid Python.

A. The Cognitive Friction Problem

The supplied SEMSYNTH design [1] frames this tension as a *Cognitive Friction Gap*. Concrete input-output (I/O) examples can constrain behavior, but a user must identify an informative input and supply its intended output. High-level Yes/No questions may offer another interaction format. Their effect on effort remains an empirical question: understanding a binary question can still require substantial reasoning. This project does not include a user study.

Consider "extract the domain from an email address." For a multilevel address, retaining the full subdomain and retaining only a registrable domain express different intentions. A clarification about subdomain treatment would make this choice explicit. This is a motivating example from the design, rather than an evaluated capability of the current string/list prototype.

B. Semantic Active Learning and Challenges

SEMSYNTH treats clarification as active learning over candidate programs. Its design selects questions expected to reduce uncertainty, updates beliefs from answers, and regenerates candidates when refinement is insufficient. The three original challenges remain: predicting how programs satisfy semantic questions; handling noisy or contradictory feedback; and managing large candidate spaces. The full design addresses them with a learned mapping model, belief revision, and tiered management. The implementation evaluated here addresses only the small-pool, deterministic-observation part.

C. Related Work

FlashFill [2] demonstrates synthesis from examples with interactive disambiguation. TiCoder [3] obtains feedback on generated tests to prune or rank code suggestions. ClarifyGPT [4] checks behavioral consistency, asks targeted questions when requirements are ambiguous, and refines the requirement before final generation. These systems motivate execution-grounded clarification, but their published performance is not used as a baseline in our experiments.

SYNTRA [5] studies test-time transduction and active elimination using predicted outputs. Reflexion [6] uses linguistic feedback across trials and motivates the regeneration component in the supplied design. ClarifyCodeBench [7] studies requirement clarification as an evaluation task. Our project does not reproduce these systems or their benchmarks; it evaluates three selectors on shared saved pools.

D. Project Scope and Contributions

The project contributes an executable subset of the supplied SEMSYNTH design, an explicit design-to-code alignment, a

controlled selection experiment, and a reproducible evidence package. It preserves the original method's 47-feature mapping, semantic information gain, atomic questions, diversity, Bayesian revision, AGA, and regeneration as the intended architecture. It does not claim these unintegrated modules as completed implementations or new research contributions.

The measured prototype executes bounded probes to obtain deterministic output partitions instead of training the 47-feature model. It implements hard filtering, records stopping reasons, and exposes the interaction through Streamlit. Optional soft updates exist in code but are outside the primary experiment. Section II retains the complete method and identifies this subset; Sections III and IV describe its measured behavior; Section V discusses cases, validity, and the remaining architecture.

II. METHOD

This section details the theoretical architecture of SEMSYNTH as originally envisioned for robust human-in-the-loop program synthesis. We first articulate the complete 7-stage canonical framework, its 47-dimensional feature space, information gain formulation, Bayesian belief revision, and adaptive regeneration. Subsequently, we formalize the *Implementation Scope* realized in our monitored engineering prototype, showing under explicit assumptions that our execution-based balanced partition scoring corresponds precisely to the deterministic limit-case of the canonical information gain metric.

A. Canonical Pipeline Overview

The complete SEMSYNTH framework operates as a structured, interactive 7-stage active learning pipeline:

1) **Specification Parsing:** Extracts functional objectives, constraints, and interface types from natural language user prompts.
2) **Feature Extraction:** Constructs a dense 47-dimensional feature representation $\mathbf{x} \in \mathbb{R}^{47}$ encoding code syntax, execution dynamics, and natural language query properties.
3) **Hypothesis Initialization (AGA):** Generates a diverse initial candidate pool $\mathcal{H}_0$ through two-stage Autoregressively Generated Algorithms.
4) **Question Selection:** Ranks candidate clarification queries $q \in \mathcal{Q}$ by combining entropy-based semantic information gain $IG(q)$ with a diversity penalty.
5) **Oracle Interaction:** Presents the optimal clarification question q^* to the user oracle, receiving a discrete feedback label $a \in \{\text{Yes}, \text{No}\}$.
6) **Belief Revision:** Updates posterior hypothesis probability weights $P(h \mid a, q)$ via contradiction-aware soft Bayesian belief updating.
7) **Termination and Adaptive Regeneration:** If belief mass concentrates on a dominant candidate, the system returns h^*; if contradictory feedback exhausts viable hypotheses, Reflexion-style adaptive regeneration is triggered.

B. 47-Dimensional Features and Mapping $M(h, q)$

In the full architecture retained from the supplied manuscript [1], predicting whether a candidate program h satisfies a natural language clarification question q without requiring exhaustive concrete execution on arbitrary domains is governed by a learned mapping function $M : \mathcal{H} \times \mathcal{Q} \to [0, 1]$, where $M(h, q) = P(h \models q)$. The model operates over a 47-dimensional feature vector $\mathbf{x}(h, q)$ composed of three structured groups:

- **18 AST Structural Features:** Node count, maximum AST depth, branching factor, loop nesting levels, external function call count, cyclomatic complexity, distinct variable identifiers, operator vocabulary diversity, Halstead volume, recursion indicators, lambda expression frequency, list comprehension presence, exception handling coverage, external import count, control flow graph diameter, generator expression flags, assertion density, and docstring presence.
- **18 Execution and Semantic Features:** Determinism score, commutativity flag, monotonicity, numerical boundedness, return type distribution, empirical runtime complexity class, I/O signature compatibility, side-effect detection, idempotency, boundary condition behavior (e.g., handling empty lists, None, or zero), error handling completeness, floating-point precision sensitivity, string encoding compliance, collection ordering invariance, in-place mutability, constraint template matching, output cardinality, and execution trace step count.
- **11 Question-Text NLP Features:** Query token length, syntactic parse depth, lexical ambiguity score, question category classifier (boundary vs. algorithmic strategy vs. structural), token overlap with specification, verb presence, negation markers, conditional clauses, universal/existential quantifiers, domain keyword overlap, and IDF-weighted specificity.

The supplied design proposes an XGBoost mapping model [8] trained on labeled code–question–answer triples. Its sub-millisecond inference objective is a design target reported in the reference, not a measurement of this prototype. No 47-feature model is trained or integrated in the evaluated course code.

C. Semantic Information Gain Pipeline

Given an active hypothesis set $\mathcal{H}_t$ with posterior belief distribution $\{w_h^{(t)}\}_{h \in \mathcal{H}_t}$ (such that $\sum_h w_h^{(t)} = 1$), the prior Shannon entropy representing hypothesis uncertainty is:

$$H(\mathcal{H}_t) = -\sum_{h \in \mathcal{H}_t} w_h^{(t)} \log_2 w_h^{(t)}. \tag{1}$$

For a proposed clarification question q, the oracle's response is a binary random variable $A \in \{\text{Yes}, \text{No}\}$. The marginal probability of observing answer $a \in \{\text{Yes}, \text{No}\}$ is given by marginalizing over all candidate hypotheses:

$$P(A = \text{Yes} \mid q) = \sum_{h \in \mathcal{H}_t} w_h^{(t)} M(h, q), \tag{2}$$

with $P(A = \text{No} \mid q) = 1 - P(A = \text{Yes} \mid q)$. Conditioned on receiving answer a, the posterior entropy of the hypothesis space is:

$$H(\mathcal{H}_t \mid A = a) = -\sum_{h \in \mathcal{H}_t} P(h \mid a, q) \log_2 P(h \mid a, q). \quad (3)$$

The expected **Semantic Information Gain** $IG(q)$ is defined as the mutual information between the hypothesis identity and the query outcome:

$$IG(q) = H(\mathcal{H}_t) - \sum_{a \in \{\text{Yes,No}\}} P(A = a \mid q) H(\mathcal{H}_t \mid A = a). \quad (4)$$

1) Atomic Questions and Template Library: The original design decomposes compound or disjunctive questions into single-proposition Yes/No queries. It describes 85 concrete templates in nine clusters: boundary, strategy, data types, I/O, higher-order operations, strings, mathematics, collection indexing (ten each), and domain-specific algorithms (five). This library and its natural-language decomposition are not integrated in the evaluated prototype. Two short semantic wording templates exist in the UI, with an I/O fallback; the primary experiment uses I/O questions only.

2) Question Diversity Regularization: To prevent asking repetitive questions probing the same semantic aspect, SEMSYNTH penalizes queries that overlap lexically or semantically with prior history $\mathcal{Q}_{\text{history}} = \{q_1, \ldots, q_{t-1}\}$:

$$IG_{\text{final}}(q) = IG(q) - \lambda \max_{q' \in \mathcal{Q}_{\text{history}}} \text{Sim}_{\text{Jaccard}}(q, q'), \quad (5)$$

The reference uses $\lambda = 0.5$ and excludes similarity above 0.85; these are retained design settings, not measured prototype parameters. Here $\text{Sim}_{\text{Jaccard}}(q, q') = \frac{|T(q) \cap T(q')|}{|T(q) \cup T(q')|}$ computes token-level set similarity and $\lambda > 0$ is a tunable diversity hyperparameter.

D. Contradiction-Aware Bayesian Belief Revision

Upon receiving oracle response a_t for query q_t, the system updates the belief weight of each hypothesis $h \in \mathcal{H}_t$ via Bayes' theorem:

$$w_h^{(t+1)} = \frac{w_h^{(t)} P(a_t \mid h, q_t)}{\sum_{h' \in \mathcal{H}_t} w_{h'}^{(t)} P(a_t \mid h', q_t)}. \quad (6)$$

In a noisy setting where the user may inadvertently give incorrect answers with probability $\epsilon \in (0, 0.5)$, the observation likelihood is smoothed symmetrically:

$$P(a_t = \text{Yes} \mid h, q_t) = (1-\epsilon) M(h, q_t) + \epsilon(1 - M(h, q_t)). \quad (7)$$

1) Consistency under Contradictions: The original design scores historical agreement as

$$s_h = \frac{1}{t} \sum_{i=1}^{t} \mathbb{I}[M(h, q_i) = a_i]. \quad (8)$$

Candidates below a consistency threshold (0.5 in the reference design) receive dampened weights instead of immediate elimination. This mechanism is intended to support recovery from conflicting feedback. Optional symmetric-noise updates in the prototype do not constitute the full consistency/recovery engine, and no noisy-feedback result is reported here.

E. Tiered Hypothesis Management

The original design first keeps a beam of top-K hypotheses by belief weight (design beam $K = 50$), and uses program-length buckets as a lightweight clustering proxy when the space is large. One representative per bucket is considered to reduce repeated work. With m representatives and a fixed question set, the intended scoring cost is $O(|\mathcal{Q}|(K + m))$, apart from feature, model, and execution costs. Length buckets are not AST tree-edit-distance clustering. The target space of up to 10^5 hypotheses belongs to the full design; this project measures pools of at most three and makes no scale or query-latency claim.

F. Autoregressively Generated Algorithms and Regeneration

AGA has two original steps: generate structurally different algorithmic strategies from the specification, then translate each strategy independently to code. The six domain hints in the supplied design are `sieve`, `dp_triangle`, `matrix_transpose`, `geometry`, `number_theory`, and `two_pointer`. Repeated direct code-generation calls in the prototype do not implement this strategy–code separation. SYNTRA [5] provides related active synthesis context, without serving as an implemented baseline here.

The design's feedback-conditioned regeneration is inspired by Reflexion [6]. Its proposed trigger is

$$\text{degenerate} \iff \max_h w_h < \epsilon_d \quad \text{or} \quad n_{\text{consecutive No}} \geq 3, \quad (9)$$

with design threshold $\epsilon_d = 0.05$, distinct from the oracle-noise parameter. The interaction history supplies `MUST-SATISFY` and `MUST-AVOID` constraints. Newly generated candidates are merged with boosted initial weights (0.3 in the design) and renormalized. These trigger and weighting choices are retained as design parameters, not tested settings of the current hard-filtering loop. The prototype stops with an empty, converged, or tentative result rather than invoking regeneration.

G. Theoretical Query Complexity

Theorem 1 (Logarithmic Query Complexity)**.** *Assume the intended hypothesis is in the initial pool, feedback and mapping are exact, and every non-singleton active set admits a question whose two parts each contain at most* $\lceil |\mathcal{H}_t|/2 \rceil$ *candidates. Then hard filtering isolates the intended hypothesis within* $\lceil \log_2 |\mathcal{H}_0| \rceil$ *questions.*

Proof. After T answers, the balanced-split assumption gives $|\mathcal{H}_T| \leq \lceil |\mathcal{H}_0| / 2^T \rceil$. For $T = \lceil \log_2 |\mathcal{H}_0| \rceil$, the bound is one. This is conditional on a distinguishing balanced question remaining available at every step; a bounded probe generator need not satisfy that assumption. □

H. Implementation Scope of the Prototype

The course implementation demonstrates active selection on a bounded pool and records its limits. Table I provides a comprehensive alignment matrix mapping each component of the canonical SEMSYNTH design to its status in our executable codebase.

1) Mathematical Derivation of the Prototype Limit-Case: Under uniform priors and deterministic observations, the prototype score is a special case of the original Information Gain equation (Eq. 4).

Consider a pool of K candidate programs $\mathcal{H} = \{h_1, \ldots, h_K\}$ evaluated over a probe input $x \in \mathcal{X}_{\text{probe}}$. Let candidate $h_i(x)$ produce observation y_i. Suppose we formulate a clarification query q: *"On input x, is the output equal to y?"*.

Under the prototype's operational assumptions:

1) **Deterministic Semantic Mapping:** The mapping $M(h_i, q) \in \{0, 1\}$ is deterministic, evaluated directly via isolated execution:
$$M(h_i, q) = \mathbb{I}[h_i(x) == y]. \tag{10}$$
2) **Uniform Prior Belief:** Prior to questioning at step t, remaining active hypotheses hold equal weight $w_h = \frac{1}{|\mathcal{H}|}$.
3) **Binary Partitioning:** The query induces a binary partition of the active set into $\mathcal{H}_{\text{yes}} = \{h \in \mathcal{H} \mid h(x) == y\}$ and $\mathcal{H}_{\text{no}} = \mathcal{H} \setminus \mathcal{H}_{\text{yes}}$.

Let $p = \frac{|\mathcal{H}_{\text{yes}}|}{|\mathcal{H}|}$ denote the fraction of hypotheses responding positively, and $1-p = \frac{|\mathcal{H}_{\text{no}}|}{|\mathcal{H}|}$. Under uniform priors, the marginal observation probabilities are:

$$P(A = \text{Yes} \mid q) = p, \quad P(A = \text{No} \mid q) = 1 - p. \tag{11}$$

When the oracle answers $A = \text{Yes}$, all hypotheses in $\mathcal{H}_{\text{no}}$ are filtered out, leaving $|\mathcal{H}_{\text{yes}}|$ equally likely candidates. Hence, the conditional entropy becomes:

$$H(\mathcal{H} \mid A = \text{Yes}) = \log_2 |\mathcal{H}_{\text{yes}}| = \log_2(p \cdot |\mathcal{H}|). \tag{12}$$

Similarly, when $A = \text{No}$:

$$H(\mathcal{H} \mid A = \text{No}) = \log_2 |\mathcal{H}_{\text{no}}| = \log_2((1 - p) \cdot |\mathcal{H}|). \tag{13}$$

Substituting these into the canonical Information Gain equation (Eq. 4) with prior entropy $H(\mathcal{H}) = \log_2 |\mathcal{H}|$:

$$\begin{aligned} IG(q) &= \log_2 n - p \log_2(pn) \\ &\quad - (1 - p) \log_2((1 - p)n) \\ &= -p \log_2 p - (1 - p) \log_2(1 - p) = H_b(p), \end{aligned} \tag{14}$$

where $n = |\mathcal{H}|$ and $H_b(p)$ is binary entropy. Only nonempty partitions are considered.

Theorem 2 (Equivalence of Information Gain and Partition Balance)**.** *Under uniform hypothesis priors and deterministic probe execution, maximizing Semantic Information Gain $IG(q)$ over the active hypothesis space is algebraically equivalent to maximizing the binary entropy $H_b(p)$ of the induced behavioral partition, attaining its global maximum at an exact 50-50 bisection ($p = 0.5$).*

Consequently, the `balanced` selector's binary score implements this special-case objective on the available probe partitions; it does not implement learned semantic mapping or diversity regularization. Furthermore, hard filtering represents the exact limiting case of Bayesian updating (Eq. 6) as the oracle noise parameter $\epsilon \to 0$.

III. EXPERIMENTAL SETUP

The evaluation asks: **RQ1**, how do the three selectors compare on the controlled candidate banks; **RQ2**, how many questions are asked and how do sessions terminate; and **RQ3**, what selection behavior is observed on the saved pools labeled as generated by two LLM backends? These are descriptive questions about a limited prototype, not tests of statistical superiority or general robustness.

A. Synthetic Cases and Pilot Catalog

The measured dataset is `config/ambiguity_benchmark.json`: 32 cases, with four variants in each of eight families. A variant contains an underspecified description, interface, seed inputs, candidate bank, reference code, and five initial evaluation inputs. The target interpretation varies across variants; for example, sorting targets may be ascending or descending. Table II lists these choices. The four variants in a family are related and are not independent real-world problems.

For the offline configuration, the runner shuffles each three-program bank using seed 3101, keeps three candidates, and saves the ordered pool. The reference implementation is available in the bank by construction. The experiment therefore evaluates selection when a useful candidate is available; it does not measure candidate-generation coverage on arbitrary tasks.

A separate file, `config/dataset_sample.json`, contains 15 pilot tasks. Eight are HumanEval-derived [9]; five match MBPP task contracts [10]; one is an adaptation of MBPP/534 and one has no verified upstream source. The pilot is used for baseline development and provenance audit, not for the 32-case result. Local tests are reduced or modified, so the catalog does not imply evaluation with either benchmark's official test suite.

The audit corrects two legacy IDs: `similar_elements` maps to MBPP/2 and `find_substring` to MBPP/91. `count_upper` is treated as a local task, since MBPP/105 counts Boolean values. The local `search_literal` returns a start index or -1, unlike MBPP/534's span result. `remove_Occ` removes the first *and* last occurrence. Upstream records, response hashes, source URLs, adaptation notes, and checks of the local tests accompany the catalog. These checks do not establish correctness on omitted upstream or unseen tests.

B. Selectors and Shared Budget

All three selectors receive the same ordered pool and the same retained evaluation inputs. DIRECT-1 chooses the first code string. FIRST-VALID chooses the first candidate admitted

TABLE I
ALIGNMENT WITH THE ORIGINAL SEMSYNTH DESIGN. IMPLEMENTATION STATUS IS DISTINCT FROM EMPIRICAL VALIDATION.

Design component	Prototype status	Implementation/evaluation boundary
Specification parsing	Partial	Description, function name and parameters; no full constraint parser.
47 features / XGBoost	Not integrated	Retained design; execution observations supply deterministic output partitions.
Candidate initialization	Partial	LLM adapters or fixed banks; strategy–code AGA is not implemented.
Information gain	Special case	Uniform priors and binary output partitions yield $H_b(p)$.
Semantic templates	Partial	Dedup/sort wording in UI; I/O fallback. Wording disabled in measured Balanced runs.
Atomic decomposition	Not integrated	Binary question form does not implement an NLP compound-question decomposer.
Diversity penalty	Not integrated	Exact repeated-question suppression only, without the Jaccard penalty.
Belief update	Partial	Hard filtering measured; optional soft weights unit-tested without noise evaluation.
Contradiction recovery	Not integrated	No full consistency/recovery pipeline in the evaluated loop.
Beam / length buckets	Not integrated	Small pools only; no large-space performance claim.
AGA / regeneration	Not integrated	No strategy stage or feedback-conditioned re-synthesis.
Oracle / termination	Implemented	Reference feedback in experiments; empty, single, budget or unresolved outcomes.
UI / evidence runner	Implemented	Streamlit, shared pools, probe separation, checkpoint/CSV and replay.

TABLE II
EIGHT RELATED TEMPLATE FAMILIES. REFERENCE INTENT VARIES ACROSS THE FOUR CASES IN EACH FAMILY.

Family	Cases	Interface	Target interpretations across variants
dedup	4	`remove_duplicates(xs)`	Stable first occurrences or sorted unique values
sorting	4	`sort_numbers(xs)`	Ascending or descending order
intersection	4	`common_elements(a, b)`	Sorted unique intersection or original-order multiplicity
parity	4	`select_numbers(xs)`	Even or odd values
case	4	`normalize_text(s)`	Lowercase or uppercase
trim	4	`clean_text(s)`	Both-side or left-only trimming
threshold	4	`is_large(x, threshold)`	Strict or inclusive lower bound
palindrome	4	`is_palindrome(s)`	Case-sensitive or case-insensitive symmetry

Every offline bank has three candidate implementations and includes its reference; each case starts with five evaluation inputs.

by the shared syntax, policy, duplicate, and top-level function-name checks. These checks do not establish full type/signature compatibility or semantic correctness. BALANCED uses binary entropy, hard filtering, at most three questions, and at most eight probes per task. Semantic wording and soft belief updates are disabled in this experiment.

Direct-1 is a selector from a pool that may have required several generation requests; it is not a separate one-call or chain-of-thought experiment. Oracle Best-of-K is not included as a deployable method because it would select using evaluation labels. A requested pool limit of three does not guarantee three distinct generated programs; the live-labeled pilot pools contain two or three after deduplication.

C. Probe Separation and Reference Oracle

Before expected outputs are computed, the runner constructs the complete bounded probe set from seed inputs and mutations and removes exact matches from the evaluation inputs. The same retained inputs are used for every selector. For 32 cases, 33 of the initial 160 input instances are excluded, leaving 127 instances, or two to five per case. Each eight-case pilot retains 31 instances. Input disjointness prevents exact reuse of a probe as an evaluation input; related patterns and the hand-authored relationship between candidates and tests remain validity limitations.

On a question proposing output y for input x, the oracle returns whether the reference program produces y on x. The reference code and evaluation labels are not passed to candidate generation. Feedback is noiseless and is supplied by code, not human participants. The small retained test sets only assess finite-suite agreement.

D. Recorded Backends and Configuration

The saved pilot manifests identify `gemini-3.5-flash-lite` through the Gemini adapter and `qwen2.5-coder:32b` through an OpenAI-compatible endpoint. Qwen2.5-Coder is described in its technical report [11]; the local tag alone does not pin a model-weight revision. These identifiers are reported as recorded, without claims about current service availability or model rankings. The primary controls are seed 3101, top-$K = 3$, question budget three, probe budget eight, five-second execution timeout, and enabled execution caching.

There is one eight-task saved run per backend and no repeated-seed study. The offline run ID is `e6a84cf2098af752`; the two pilot IDs are `8be202264a584167` and `b6c988199ed7b1d5`. Their complete manifests, pools, checkpoints, and CSV files are included. Original live generation is not repeated during final verification: selection and evaluation are replayed from saved code. Provider request IDs, token usage, generation latency, and a pinned Qwen weight digest were not recorded, so exact regeneration of the original pools is outside the reproducibility claim.

TABLE III
PILOT PROVENANCE AUDIT (15 TASKS). THESE TASKS ARE NOT THE MEASURED 32-CASE BENCHMARK.

Local ID	Legacy ID	Audited source	Contract/adaptation boundary
HE_001	HumanEval/0	HumanEval/0	Source contract checked; local tests reduced/adapted.
HE_002	HumanEval/15	HumanEval/15	Source contract checked; local tests reduced/adapted.
HE_003	HumanEval/28	HumanEval/28	Source contract checked; local tests reduced/adapted.
HE_004	HumanEval/14	HumanEval/14	Source contract checked; local tests reduced/adapted.
HE_005	HumanEval/43	HumanEval/43	Source contract checked; local tests reduced/adapted.
HE_006	HumanEval/54	HumanEval/54	Source contract checked; local tests reduced/adapted.
HE_007	HumanEval/58	HumanEval/58	Source contract checked; local tests reduced/adapted.
HE_008	HumanEval/62	HumanEval/62	Source contract checked; local tests reduced/adapted.
MB_001	MBPP/1	MBPP/2	ID corrected; tuples represented as JSON lists.
MB_002	MBPP/8	MBPP/8	Source contract checked; local tests reduced/adapted.
MB_003	MBPP/11	MBPP/11	Removes first and last occurrences.
MB_004	MBPP/56	MBPP/91	ID corrected; local parameter name differs.
MB_005	MBPP/62	MBPP/62	Source contract checked; local tests reduced/adapted.
MB_006	MBPP/105	Local only	Local ASCII-uppercase task; no source verified.
MB_007	MBPP/119	MBPP/534	MBPP/534 adaptation: integer index instead of span.

Thirteen upstream contracts are checked against local tests; two local/adapted contracts are kept separate. URLs, licenses, hashes and 46 local-test checks are included in the CSV and audit evidence.

TABLE IV
CONTROLLED OFFLINE RESULTS ON 32 CASES. RATE IS FINITE-SUITE CASE SUCCESS.

Method	Passed	Rate	Mean Qs
Direct-1	13/32	40.625%	0.0000
First-valid	13/32	40.625%	0.0000
Balanced	32/32	100.000%	1.4688

TABLE V
BALANCED STOPPING STATES. ALL LISTED SELECTIONS PASS RETAINED TESTS; UNRESOLVED REMAINS TENTATIVE.

Run (cases)	One remains	Unresolved
Offline (32)	29	3
Gemini (8)	2	6
Qwen (8)	3	5

E. Metrics and Verification

A case passes when its selected program passes every retained evaluation input. We report passed cases/total cases, question count, and termination reason separately. Having one candidate means convergence within a finite pool, not correctness for all inputs. Per-method elapsed times exclude pool generation and human response time, so no end-to-end latency or cost claim is made. Final verification checks the software tests, recomputes saved selection outcomes, compares them with CSV/checkpoint fields, and builds the report from unpacked source in a fresh environment.

IV. RESULTS

A. Controlled Offline Selection (RQ1)

Table IV reports the 32-case controlled run. Direct-1 and First-valid each pass 13 cases (40.625%); Balanced passes 32 (100% of the retained finite test suites). The observed difference is 19 cases, or 59.375 *percentage points*. On individual retained inputs, the noninteractive selectors pass 71/127 and Balanced passes 127/127. The case-level result remains primary.

Both noninteractive rows are identical because admission checks cannot resolve the intended interpretation among these admissible programs. The first-entry baseline uses shuffled hand-authored code, not an observed LLM output distribution. Its failure fraction therefore does not estimate a language model's generation error rate.

Family-level results are descriptive: Direct-1 passes 3/4 in case conversion and trimming, 2/4 in duplicate removal and palindrome checking, 1/4 in intersection, parity, and sorting, and 0/4 in threshold filtering. Balanced passes 4/4 in each family. Four correlated cases per family are insufficient for a general domain-difficulty ranking or a statistical superiority claim.

B. Questions and Termination (RQ2)

Balanced asks 47 questions in total, averaging 1.4688 per case: 17 cases use one question and 15 use two. No case reaches the three-question limit. This is compatible with small informative partitions, but it does not verify the assumptions of Theorem 1. The theorem requires distinguishability throughout refinement; it does not guarantee a useful question within every bounded probe set.

Twenty-nine offline sessions finish with one active candidate. Three stop with `no_distinguishing_question`; their chosen programs still pass the retained tests. These programs are indistinguishable on the available probes, not proven equivalent on all valid inputs. The interface must retain the tentative status rather than treat test success as universal correctness.

C. Saved LLM Pilot Pools (RQ3)

Table VI reports eight synthetic cases per recorded backend. For the Gemini-labeled run, Direct-1 and First-valid pass 7/8 while Balanced passes 8/8, an observed difference of one case (12.5 percentage points). For the Qwen-labeled run, the corresponding results are 6/8 and 8/8, a difference of two cases

(25 percentage points). Mean question counts are 0.25 and 0.50.

The baseline failures in these saved runs occur on parity for Gemini and on duplicate removal and parity for Qwen. Balanced selects a passing alternative already present in each pool. It does not repair source code or generate a missing correct candidate. Low question counts partly reflect limited observable diversity: six Gemini sessions and five Qwen sessions stop without a distinguishing question. Only two and three sessions respectively converge to one candidate (Table V).

All pilot selections pass their retained tests, including unresolved sessions. With eight related cases, one ordering/seed, noiseless feedback, and no model-generation replication, these observations support a small feasibility demonstration. They do not establish performance on independent tasks, superiority to published interactive systems, noise tolerance, or human cognitive benefits.

D. Evidence Consistency

The saved artifacts contain 96 offline task-method outcomes, 24 for each pilot, and three for each of two smoke runs. Replay recomputes all 150 outcomes from saved pools and checks pass counts, selected-code hashes, question counts, candidate counts, excluded inputs, and stopping states. Timing fields are deliberately not compared because platform and load vary. The same outputs validate the numerical tables; replay does not independently authenticate historical provider responses.

V. DISCUSSION AND LIMITATIONS

A. Cases from Saved Artifacts

1) Selecting an Available Alternative: In the Qwen-labeled pilot, `SYN_DEDUP_01` contains two saved candidates after deduplication. The first returns `list(set(xs))`; the second uses a `seen` set and appends first occurrences to an ordered result. On `[4,1,4,2,1]`, the first produces `[1,2,4]` in the recorded Python environment, while the second produces `[4,1,2]`. The reference intention is to preserve first occurrences.

Replaying the saved pool asks whether the first output, `[1,2,4]`, is intended on that input. The reference oracle answers No, eliminating the first candidate. One remains and passes 4/4 retained evaluation inputs. This is selection among existing alternatives, not code repair. The question, answer, partition identifiers, and chosen-code hash are preserved in the replay evidence.

2) Passing Tests Without Convergence: The Gemini-labeled `SYN_DEDUP_01` session asks no questions, stops with several candidates and no distinguishing probe, and passes its retained tests. Syntactically different implementations can agree on the available inputs. This observation only describes the bounded probe/evaluation sets. For example, dictionary-based duplicate removal and an equality-based loop can differ on unhashable values; no unrestricted equivalence claim follows from small integer-list tests.

3) Observed Baseline Failures and Coverage Limits: The saved Gemini parity pool begins with a majority-parity policy, whereas the reference for this variant selects even values. Balanced uses one question to select a passing alternative. The Qwen pilot contains two first-entry failures, on duplicate removal and parity. These are observed cases, not a general failure-frequency taxonomy.

A separate architectural limitation is pool coverage: if no candidate satisfies the intended behavior, filtering cannot synthesize a missing solution. This is a hypothetical failure mechanism, not an additional measured case in the 32-case offline run whose reference is in each bank. AGA and feedback-conditioned regeneration remain relevant extensions of the original design, but their benefits have not been measured in this project.

B. Interactive Implementation

The Streamlit interface supports offline fixtures and provider adapters, displays a concrete binary question, records answers and filtering changes, and exposes tentative outcomes. Offline UI captures are supplied in the source artifact; they illustrate the interface rather than establish LLM generation quality or human usability. List-style parameter names such as `xs, ys` are needed for the current default type heuristic; names `a, b` may be inferred as numbers without explicit seed inputs.

Execution uses AST policy checks, subprocesses, UTF-8 communication, timeout, and cache. This is local process isolation, not a hardened multi-tenant sandbox. The software suite exercises execution, candidate admission, filtering, bookkeeping, and Streamlit behavior; software-test success is separate from synthesis test success.

C. Threats to Validity

Internal validity. Candidate banks, reference programs, and test inputs are hand-authored and related. Exact probe overlap is removed, but structural similarity remains. Historical live-generation request IDs and token logs are unavailable; only saved-pool selection and evaluation can be re-executed exactly under the recorded protocol. The API model tag and private endpoint do not pin service implementation or weights.

External validity. Thirty-two variants represent only eight small families, and each live-labeled pilot contains one variant per family. All tasks are standalone functions and use one selection seed. Generalization to independent tasks, larger codebases, third-party libraries, or other backends is unmeasured.

Construct validity. Passing two to five retained inputs is not a proof of program correctness. Single-candidate convergence and finite-suite pass rate are different outcomes. Question count is an interaction-cost proxy, not user effort. No user study, noisy-oracle experiment, large-space scalability test, token-cost comparison, or significance test is claimed.

D. Roadmap toward the Full Design

First, extend independent task coverage and the provenance protocol. Next integrate the 47-feature extractor and train/calibrate $M(h, q)$, then evaluate atomic questions and

TABLE VI
SAVED LLM-LABELED PILOT POOLS: EIGHT SYNTHETIC CASES PER RECORDED BACKEND, REQUESTED TOP-$K = 3$.

Recorded model identifier	Selector	Passed	Rate	Mean Qs
`gemini-3.5-flash-lite`	Direct-1	7/8	87.5%	0.00
`gemini-3.5-flash-lite`	First-valid	7/8	87.5%	0.00
`gemini-3.5-flash-lite`	Balanced	8/8	100.0%	0.25
`qwen2.5-coder:32b`	Direct-1	6/8	75.0%	0.00
`qwen2.5-coder:32b`	First-valid	6/8	75.0%	0.00
`qwen2.5-coder:32b`	Balanced	8/8	100.0%	0.50

Same ordered pool per task; actual saved pools contain two or three programs. Reference-program feedback is noiseless.

diversity with candidate pools held fixed. Belief consistency and recovery require controlled noise studies. AGA and constraint-conditioned regeneration require coverage/regeneration experiments. Finally, tiered management needs scale measurements, and semantic wording needs a user assessment. These are future milestones rather than missing requirements for this partial course implementation.

VI. CONCLUSION

This project preserves the supplied SEMSYNTH architecture and implements a limited executable subset: candidate adapters, bounded probes, binary partition entropy, hard filtering, and explicit stopping states. The learned 47-feature mapping, atomic/diversity machinery, AGA, scalable hierarchy, and adaptive regeneration remain part of the design and future integration work.

On 32 controlled synthetic cases, Balanced passes every retained test in 32 cases versus 13 for each noninteractive selector, using 1.4688 questions per case. Two saved eight-case LLM-labeled pilots show 8/8 passing selections versus 7/8 and 6/8 respectively. These finite-suite results establish the observed behavior of selection with a noiseless reference oracle and the available pools. They leave candidate-generation coverage, general correctness, noisy feedback, scalability, and human effort open. The source and evidence package supports replay of the measured selection outcomes and a clean rebuild of this report.

ACKNOWLEDGMENT AND CONTRIBUTIONS

The project is supervised by TS. Võ Thanh Hùng. Repository history attributes the initial Lite scaffold, problem framing, and pilot preparation to Nguyễn Thành Đạt; the phase-one clarification text and flow diagram to Nguyễn Tiến Vũ Bảo; and the executable pipeline, provider adapters, interface, tests, and experiment infrastructure to Đặng Bá Ty. Subsequent integration, audit, experiments, and report preparation build on these artifacts. This account describes visible repository work and does not quantify contributions outside Git.

REFERENCES

[1] SEMSYNTH project manuscript, "Semantic Information Gain for Robust Human-in-the-Loop Program Synthesis," 2026, unpublished design manuscript supplied with the project; original source retained in report/reference. Not presented as a published ICSE paper.

[2] S. Gulwani, "Automating String Processing in Spreadsheets using Input-Output Examples," in *Proceedings of POPL*, 2011. [Online]. Available: https://www.microsoft.com/en-us/research/publication/automating-string-processing-spreadsheets-using-input-output-examples/

[3] S. K. Lahiri, S. Fakhoury, A. Naik *et al.*, "Interactive Code Generation via Test-Driven User-Intent Formalization," *arXiv preprint arXiv:2208.05950*, 2022. [Online]. Available: https://arxiv.org/abs/2208.05950

[4] F. Mu, L. Shi, S. Wang *et al.*, "ClarifyGPT: Empowering LLM-based Code Generation with Intention Clarification," *arXiv preprint arXiv:2310.10996*, 2023. [Online]. Available: https://arxiv.org/abs/2310.10996

[5] K.-i. Lee, J. Koo, S. Yoon *et al.*, "Program Synthesis via Test-Time Transduction," *arXiv preprint arXiv:2509.17393*, 2025. [Online]. Available: https://arxiv.org/abs/2509.17393

[6] N. Shinn, F. Cassano, E. Berman *et al.*, "Reflexion: Language Agents with Verbal Reinforcement Learning," *arXiv preprint arXiv:2303.11366*, 2023. [Online]. Available: https://arxiv.org/abs/2303.11366

[7] Z. Fang, D. Jin, Y. dong *et al.*, "ClarifyCodeBench: Evaluating LLMs on Clarifying Ambiguous Requirements for Code Generation," *arXiv preprint arXiv:2607.00711*, 2026. [Online]. Available: https://arxiv.org/abs/2607.00711

[8] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," *arXiv preprint arXiv:1603.02754*, 2016. [Online]. Available: https://arxiv.org/abs/1603.02754

[9] M. Chen, J. Tworek, H. Jun *et al.*, "Evaluating Large Language Models Trained on Code," *arXiv preprint arXiv:2107.03374*, 2021. [Online]. Available: https://arxiv.org/abs/2107.03374

[10] J. Austin, A. Odena, M. Nye *et al.*, "Program Synthesis with Large Language Models," *arXiv preprint arXiv:2108.07732*, 2021. [Online]. Available: https://arxiv.org/abs/2108.07732

[11] B. Hui, J. Yang, Z. Cui *et al.*, "Qwen2.5-Coder Technical Report," *arXiv preprint arXiv:2409.12186*, 2024. [Online]. Available: https://arxiv.org/abs/2409.12186